

HDOC Day-34 — Variable-Length Square Matrix Operations

1. Algorithm

Step 1: Start the program.

Step 2: Read the order n of the square matrix.

Step 3: Declare a variable-length 2D array `a[n][n]`.

Step 4: Read all matrix elements.

Step 5: Repeatedly display the following menu:

1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Display diagonal traversal
4. Exit

Step 6: Read the user's choice and use `switch-case`.

For Choice 1 — Distinct Main Diagonal

1. Compare every main diagonal element `a[i][i]` with the remaining diagonal elements `a[j][j]`.
2. If any two elements are equal, they are **not distinct**.
3. Otherwise, all main diagonal elements are **distinct**.

For Choice 2 — Sum of Main Diagonal

1. Initialize `sum = 0`.
2. Traverse from `i = 0` to `n-1`.
3. Add `a[i][i]` to `sum`.
4. Display the sum.

For Choice 3 — Diagonal Traversal

1. There are $2n - 1$ diagonals in an $n \times n$ matrix.
2. Start from the top-left element.
3. Traverse successive diagonals alternately:
 - upward-right
 - downward-left
4. Display each element during traversal.

For Choice 4: Exit the program.

Default: Display "Invalid choice!".

Step 7: Stop.

2. C Program

```
#include <stdio.h>

// Function to check whether main diagonal elements are distinct
void checkDistinct(int n, int a[n][n])
{
    int i, j, distinct = 1;

    printf("Main diagonal elements: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i][i]);

    for (i = 0; i < n; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (a[i][i] == a[j][j])
            {
                distinct = 0;
                break;
            }
        }

        if (distinct == 0)
            break;
    }

    if (distinct)
        printf("\nAll the main diagonal elements are distinct.\n");
    else
        printf("\nAll the main diagonal elements are not distinct.\n");
}

// Function to find sum of main diagonal elements
void diagonalSum(int n, int a[n][n])
{
    int i, sum = 0;

    for (i = 0; i < n; i++)
        sum = sum + a[i][i];

    printf("Sum of main diagonal elements: %d\n", sum);
}

// Function for diagonal traversal
void diagonalTraversal(int n, int a[n][n])
{
    int d, i, j;

    printf("Diagonal traversal: ");

    for (d = 0; d <= 2 * n - 2; d++)
```

```

{
    if (d % 2 == 0)
    {
        // Downward-left direction
        if (d < n)
        {
            i = 0;
            j = d;
        }
        else
        {
            i = d - n + 1;
            j = n - 1;
        }

        while (i < n && j >= 0)
        {
            printf("%d ", a[i][j]);
            i++;
            j--;
        }
    }
    else
    {
        // Upward-right direction
        if (d < n)
        {
            i = d;
            j = 0;
        }
        else
        {
            i = n - 1;
            j = d - n + 1;
        }

        while (i >= 0 && j < n)
        {
            printf("%d ", a[i][j]);
            i--;
            j++;
        }
    }
}

printf("\n");
}

// Main function
int main()
{
    int n, i, j, choice;

    printf("Enter the order of square matrix: ");
    scanf("%d", &n);

    int a[n][n];

```

```

printf("Enter the matrix elements:\n");

for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        scanf("%d", &a[i][j]);
    }
}

do
{
    printf("\n--- MENU ---\n");
    printf("1. Check main diagonal elements are distinct\n");
    printf("2. Find sum of main diagonal elements\n");
    printf("3. Display diagonal traversal\n");
    printf("4. Exit\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            checkDistinct(n, a);
            break;

        case 2:
            diagonalSum(n, a);
            break;

        case 3:
            diagonalTraversal(n, a);
            break;

        case 4:
            printf("Exiting program...\n");
            break;

        default:
            printf("Invalid choice! Please enter 1 to 4.\n");
    }

} while (choice != 4);

return 0;
}

```

3. Test Case Table

Test Case	Matrix	Choice	Expected Output
TC-1	1 2 3 / 4 5 6 / 7 8 9	1	Main diagonal 1 5 9 → All distinct

Test Case	Matrix	Choice	Expected Output
TC-2	1 2 3 / 4 1 6 / 7 8 9	1	Main diagonal 1 1 9 → Not all distinct
TC-3	1 2 3 / 4 5 6 / 7 8 9	2	Sum = 1+5+9 = 15
TC-4	1 2 3 / 4 5 6 / 7 8 9	3	1 2 4 7 5 3 6 8 9
TC-5	5	1, 2, 3	Distinct; Sum = 5; Traversal = 5
TC-6	Any valid matrix	7	Invalid choice!
TC-7	Any valid matrix	4	Program exits

4. Test Case Simulation and Evaluation

Test Case 1 — Distinct Main Diagonal

```

(kali@kali)-[~/Desktop]
$ cc day34.c

(kali@kali)-[~/Desktop]
$ ./a.out
Enter the order of square matrix: 3
Enter the matrix elements:
1 2 3
4 5 6
7 8 9

— MENU —
1. Check main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Display diagonal traversal
4. Exit
Enter your choice: 1
Main diagonal elements: 1 5 9
All the main diagonal elements are distinct.

```

Evaluation: The function compares every main diagonal element with the remaining diagonal elements. Since no duplicate is found among 1, 5, 9, the program correctly determines that all diagonal elements are distinct. **Test case passed.**

Test Case 2 — Repeated Main Diagonal Element

```
Enter the order of square matrix: 3
Enter the matrix elements:
1 2 3
4 1 6
7 8 9

— MENU —
1. Check main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Display diagonal traversal
4. Exit
Enter your choice: 1
Main diagonal elements: 1 1 9
All the main diagonal elements are not distinct.
```

Evaluation: The program successfully detects the duplicate value 1 on the main diagonal and reports that the diagonal elements are not distinct. **Test case passed.**

Test Case 3 — Main Diagonal Sum

```
Enter the order of square matrix: 3
Enter the matrix elements:
1 2 3
4 5 6
7 8 9

— MENU —
1. Check main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Display diagonal traversal
4. Exit
Enter your choice: 2
Sum of main diagonal elements: 15
```

Evaluation: The function accesses only the elements where the row index and column index are equal and correctly calculates their sum as 15. **Test case passed.**

Result

The menu-driven C program was successfully implemented using **separate functions and a variable-length square 2D array** to check distinct main diagonal elements, calculate their sum, and perform alternating diagonal traversal.